

Notification System & Outbox Pattern

Interview Q&A — Cases solved for reliable job enqueue (DB + Redis) | SDE-1 / Backend

Pipeline covered:

```
API validate
! ' DB transaction: Notification + Outbox (PENDING)
! ' Outbox relay (interval): queue.add(jobId)
! ' success / already exists ! ' Outbox PROCESSED
! ' Redis fail ! ' leave PENDING, retry
```

Q1. Server crashes after DB save, before Redis queue.add — what happens?

Problem

Two separate steps: (1) save Notification PENDING in Postgres, (2) queue.add to Redis. If the process dies between them, the row stays PENDING forever, no job in Redis, and if outbox is only written after queue failure, there is no outbox either. User never gets notified.

Solution

Use the transactional outbox pattern:

- 1) In one DB transaction insert Notification (PENDING) and Outbox (PENDING).
- 2) Commit.
- 3) Background outbox relay reads Outbox PENDING !' queue.add !' marks Outbox PROCESSED.

After commit, even if the server dies, the Outbox row remains and the relay enqueues later.

Follow-up 1: If the server dies before commit, does outbox still save you?

No — and that is correct. Before commit both inserts roll back. Nothing is saved. The client should retry the API. You avoid a half-written state (notification without outbox).

Follow-up 2: Can you put queue.add inside the same DB transaction?

No. A Postgres transaction only covers the database. Redis is a different system and cannot join that transaction. Correct order: commit DB (notification + outbox) first, then enqueue (directly or via relay).

Follow-up 3: Why not scan all Notification rows with status PENDING instead of Outbox?

PENDING on Notification means “not delivered to the user yet,” not “not pushed to the queue.” A job already in Redis waiting to send is still PENDING. Scanning it again creates duplicate jobs. Outbox PENDING means specifically “not yet published to the queue.”

Q2. What if Redis is down when we try to enqueue?

Problem

If you only call queue.add after saving the notification, Redis downtime means the job never enters the queue. Without a durable “must enqueue” record, the notification is stuck.

Solution

Always write Outbox in the same transaction as Notification.

When Redis is down: relay queue.add fails !' do NOT mark PROCESSED !' leave PENDING !' retry on next interval when Redis is back.

API can still return “accepted” after DB commit because intent is durable in Postgres.

Follow-up 1: Should the API return failure if Redis is down but DB commit succeeded?

Prefer accepting the request (e.g. 202) if Notification + Outbox are committed. Delivery is async. Returning 500 after durable accept confuses clients and causes duplicate retries.

Follow-up 2: Is “write outbox only when queue.add fails” enough?

No. That is a fallback, not a full outbox. Crash between DB save and queue/outbox write still loses enqueue intent. Always write outbox with the notification; treat Redis as best-effort + relay.

Q3. What if the database is down?

Problem

You cannot persist Notification or Outbox.

Solution

Fail the request (500/503). Do not pretend the notification was accepted. There is no durable record to retry from.

Follow-up 1: What if DB works for Notification but fails on Outbox insert?

With a real transaction, both roll back and the client retries. Without a transaction you can get Notification without Outbox. Fix: `prisma.$transaction` and throw on failure (do not return `{ success: false }` inside the txn).

Follow-up 2: Why must repository throw inside a transaction instead of returning `{ success: false }`?

Returning a normal object looks like success to Prisma's transaction callback, so it may COMMIT. Throwing aborts and ROLLS BACK both writes.

Q4. Explain the outbox relay. Is it a BullMQ Worker?

Problem

People confuse BullMQ Worker (pulls jobs FROM Redis and sends email/SMS) with Outbox relay (scans Postgres Outbox and calls `queue.add`).

Solution

Outbox relay is NOT new Worker(...). It is a scheduled loop:

```
every few seconds:
  find Outbox where status = PENDING
  for each row:
    queue.add(notificationId, { jobId })
    if success !' mark PROCESSED
    if fail !' leave PENDING
```

Reuse the existing Queue + Redis connection. Do not put Worker in `queue.js` and import it everywhere (consumer would start as a side effect on import).

Follow-up 1: How do you run the relay on an interval?

`setInterval` from `server.js` for a lab, or cron / separate process in production. Start it when the app boots; the file alone does nothing.

Follow-up 2: Why can `setInterval` overlap even every 3 seconds?

`setInterval` fires on the clock and does not wait for the previous async run to finish. If one scan takes 10s, the next tick at 3s starts another run. Fix with an `isRunning` lock: if running return; else set true, await worker, finally set false.

Follow-up 3: Why not `forEach` with `await` for DB/Redis updates?

`forEach` does not await async callbacks. It fires iterations quickly and does not wait. Use `for...of` + `await` so each row completes add !' update before the next.

Q5. `queue.add` succeeds but Outbox PROCESSED update fails (DB crash). Then what?

Problem

Job is in Redis, Outbox still PENDING. Next tick tries add again !' duplicate risk. Redis and Postgres are not one transaction (at-least-once enqueue intent).

Solution

- 1) Idempotent jobld: `queue.add(..., { jobld: `notification-${id}` })`. Second add with same jobld does not create a second job.
- 2) Treat “already exists” as success and mark Outbox PROCESSED (see Q6).
- 3) Send worker should also be idempotent (skip if already SENT).

Follow-up 1: Can we get exactly-once across DB + Redis?

Practically no across two systems. Aim for at-least-once + idempotency.

Follow-up 2: What does jobld mean in simple words?

A stable passport for the job in Redis. Same passport twice !' BullMQ rejects duplicate instead of creating two jobs.

Q6. Duplicate jobld makes Outbox stuck PENDING forever — why? Fix?

Problem

Job already in Redis with jobld notification-101, Outbox still PENDING. Next tick: `queue.add` throws “already exists” !' your catch returns `{ success: false }` !' relay only marks PROCESSED on success true !' Outbox stays PENDING !' fail loop every 3s. jobld blocked the duplicate, but your code treated that as failure.

Solution

In catch, distinguish errors:

```
if message includes "already exists":
  return { success: true } // already queued !' enqueue done
else:
  return { success: false } // Redis down !' retry later
```

Do NOT return success true for every catch — that would mark PROCESSED when Redis is actually down.

Follow-up 1: Why is `if (!response)` after `queue.add` useless?

On failure BullMQ throws; it does not return a falsy value. Soft-fail handling belongs in catch, or in your wrapper's `{ success }` object.

Follow-up 2: When do I use `throw` vs `if (response.success)`?

Prisma/BullMQ/Redis fail !' usually auto-throw !' try/catch. Your helper returns `{ success }` !' if `(response.success)`. Business rule inside transaction !' throw new Error to rollback. `if (!response)` after add/create is usually useless for these libraries.

Q7. Validation hang — invalid body, request never returns

Problem

Middleware only called `next()` when valid. On invalid body: no `next()`, no `res.status()`. Express leaves the request open !' client hangs. Also `return { error }` does not send an HTTP response.

Solution

```
safeParse
  if success !' return next()
  else !' return res.status(400).json({ msg, errors })
catch !' return res.status(500).json(...)
```

Must return `next()` so you do not also send 400 after a valid request (double response / headers already sent).

Follow-up 1: What status codes for accept vs validation vs server error?

Validation fail !' 400. Durable accept (async notify) !' 202 (or 200). Unexpected / DB down !' 500/503.

Q8. End-to-end: Design how you reliably enqueue a notification job

Problem

Interviewer wants a complete reliability story across API, DB, Redis, and background jobs.

Solution

- 1) Validate input (400 on fail; never hang).
- 2) One DB transaction: insert Notification + Outbox; commit; return accepted.
- 3) Outbox relay (interval + overlap lock): PENDING !' queue.add with stable jobId; success or already exists !' PROCESSED; Redis down !' leave PENDING.
- 4) Separate BullMQ Worker later: consume queue !' send email/SMS !' mark Notification SENT/FAILED (idempotent).

Failure modes covered: crash after commit, Redis down, DB down on accept, duplicate enqueue, interval overlap, forEach async mistakes.

Follow-up 1: What is still missing after enqueue works?

The send worker (BullMQ Worker), retries/backoff, dead-letter queue, provider failures, and idempotent send so the user does not get duplicate emails.

Follow-up 2: Why separate Outbox relay and send Worker?

Different responsibilities: Relay = DB !' Redis (publish intent). Worker = Redis !' email provider (deliver). Mixing them couples "get into queue" with "send email" and makes retries harder to reason about.

Quick cheat sheet

- DB down on accept !' fail request
- Crash before commit !' rollback; client retries
- Crash after commit !' Outbox remains; relay enqueues
- Redis down !' Outbox stays PENDING; retry
- Add OK, PROCESSED update fails !' jobId + treat duplicate as success
- Interval overlap !' isRunning lock
- Invalid API body !' 400; always respond
- Async loops !' for...of + await (not forEach)

SDE-1 takeaway

Knowing dual-write pain + transactional outbox is strong. Implementing transactional write + relay + jobId + overlap lock is interview-ready for the enqueue path. Expect follow-ups on send-worker idempotency and DLQ for the full notification system.

Generated from backend-engineering-labs notification-service learning session.